

Scanner Implementation

I decided to implement all the keywords, identifiers, and operators from the lox language; however I added a few of my own.

- Additions to CN language:
 - Operator: MODULO (%)
 - Keyword: ELSEIF
 - Keyword: BREAK
 - Keyword: CONST
 - I also switched the NIL Keyword to NULL, as I like that wording better

The CONST keyword I plan to implement as a variable that cannot be reassigned or altered after assignment.

Extra Credit Opportunities

Nested Block Comments

- I implemented nested block comments. I implemented my comments with the # character. Multi line and nested comments begin with `#*` and end with `*#`. I have specific test cases denoted to testing the nested block comments. Nested block comments are a valuable addition for convenience and flexibility. During development, large blocks of code can be commented out without impacting outer comments. These nested comments also demonstrate that the CN language can identify and match characters at different depths, regardless of how many levels.

Other Comments

In `cn_token.py`, the file where I create the Token class, the `str` function includes a portion of AI code. I had written an f string myself that was sufficient, but I wanted my output to be formatted better to identify the tokens while testing. I had ChatGPT write me this code that includes padding so I can see the token type, lexeme, literal, and line easier.

Error Handling

- Unterminated Strings
- Unexpected Characters
- Unterminated Nested Block Comments

Lexical Grammar Design

- Number: `[0-9]+(\.[0-9]+)?`
 - My scanner does not except .0 or 0. as numbers, there needs to be a number on both sides of the decimal point
- String: `"([^\"]*)"`
- Identifier: `[A-Za-z_][A-Za-z0-9_]*`

Test Cases

All test files can be found in the test/Lab1 folder.

I apologize as the test cases are quite lengthy. I have worked as an intern for a Quality Assurance team for the past two summers now, and I primarily create test cases in my job. My brain is wired for thorough testing!

TC1 – Keywords

Purpose

- Ensure each keyword scans to the correct token

Expected Output:

Token(type=TokenType.AND	, lexeme='and'	, literal=None	, line=1)
Token(type=TokenType.CLASS	, lexeme='class'	, literal=None	, line=2)
Token(type=TokenType.ELSE	, lexeme='else'	, literal=None	, line=3)
Token(type=TokenType.FALSE	, lexeme='false'	, literal=None	, line=4)
Token(type=TokenType.FOR	, lexeme='for'	, literal=None	, line=5)
Token(type=TokenType.FUN	, lexeme='fun'	, literal=None	, line=6)
Token(type=TokenType.IF	, lexeme='if'	, literal=None	, line=7)
Token(type=TokenType.ELSEIF	, lexeme='elseif'	, literal=None	, line=8)
Token(type=TokenType.NULL	, lexeme='null'	, literal=None	, line=9)
Token(type=TokenType.OR	, lexeme='or'	, literal=None	, line=10)
Token(type=TokenType.PRINT	, lexeme='print'	, literal=None	, line=11)
Token(type=TokenType.RETURN	, lexeme='return'	, literal=None	, line=12)
Token(type=TokenType.SUPER	, lexeme='super'	, literal=None	, line=13)
Token(type=TokenType.THIS	, lexeme='this'	, literal=None	, line=14)

```
Token(type=TokenType.TRUE      , lexeme='true'      , literal=None      , line=15)
Token(type=TokenType.VAR      , lexeme='var'      , literal=None      , line=16)
Token(type=TokenType.WHILE    , lexeme='while'    , literal=None      , line=17)
Token(type=TokenType.CONST    , lexeme='const'    , literal=None      , line=18)
Token(type=TokenType.BREAK    , lexeme='break'    , literal=None      , line=19)
Token(type=TokenType.EOF      , lexeme=''         , literal=None      , line=19)
```

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC1-Keywords.cn
Token(type=TokenType.AND      , lexeme='and'      , literal=None      , line=1)
Token(type=TokenType.CLASS    , lexeme='class'    , literal=None      , line=2)
Token(type=TokenType.ELSE     , lexeme='else'     , literal=None      , line=3)
Token(type=TokenType.FALSE    , lexeme='false'    , literal=None      , line=4)
Token(type=TokenType.FOR      , lexeme='for'      , literal=None      , line=5)
Token(type=TokenType.FUN      , lexeme='fun'      , literal=None      , line=6)
Token(type=TokenType.IF       , lexeme='if'       , literal=None      , line=7)
Token(type=TokenType.ELSEIF   , lexeme='elseif'   , literal=None      , line=8)
Token(type=TokenType.NULL     , lexeme='null'     , literal=None      , line=9)
Token(type=TokenType.OR       , lexeme='or'       , literal=None      , line=10)
Token(type=TokenType.PRINT    , lexeme='print'    , literal=None      , line=11)
Token(type=TokenType.RETURN   , lexeme='return'   , literal=None      , line=12)
Token(type=TokenType.SUPER    , lexeme='super'    , literal=None      , line=13)
Token(type=TokenType.THIS     , lexeme='this'     , literal=None      , line=14)
Token(type=TokenType.TRUE     , lexeme='true'     , literal=None      , line=15)
Token(type=TokenType.VAR      , lexeme='var'      , literal=None      , line=16)
Token(type=TokenType.WHILE    , lexeme='while'    , literal=None      , line=17)
Token(type=TokenType.CONST    , lexeme='const'    , literal=None      , line=18)
Token(type=TokenType.BREAK    , lexeme='break'    , literal=None      , line=19)
Token(type=TokenType.EOF      , lexeme=''         , literal=None      , line=19)
```

TC2 – Multi_Line_Keywords

Purpose

- Ensure that multiple keywords on the same line are scanned correctly
- Ensure keywords that lack spacing produce an IDENTIFIER token rather than the keyword token (ex. andand is an IDENTIFIER rather than two AND tokens)
- Verify that newlines and tabs are not consumed by the scanner and do not produce a token

Expected Output:

```
Token(type=TokenType.IDENTIFIER, lexeme='andandand' , literal=None , line=1)
Token(type=TokenType.FALSE    , lexeme='false'    , literal=None , line=3)
Token(type=TokenType.TRUE     , lexeme='true'     , literal=None , line=3)
Token(type=TokenType.BREAK    , lexeme='break'    , literal=None , line=3)
```

```

Token(type=TokenType.RETURN      , lexeme='return'      , literal=None      , line=5)
Token(type=TokenType.SUPER       , lexeme='super'       , literal=None       , line=5)
Token(type=TokenType.FUN         , lexeme='fun'         , literal=None       , line=5)
Token(type=TokenType.IF          , lexeme='if'          , literal=None       , line=5)
Token(type=TokenType.ELSE        , lexeme='else'        , literal=None       , line=5)
Token(type=TokenType.ELSE        , lexeme='else'        , literal=None       , line=7)
Token(type=TokenType.IF          , lexeme='if'          , literal=None       , line=7)
Token(type=TokenType.ELSEIF      , lexeme='elseif'      , literal=None       , line=7)
Token(type=TokenType.NULL        , lexeme='null'        , literal=None       , line=9)
Token(type=TokenType.THIS        , lexeme='this'        , literal=None       , line=9)
Token(type=TokenType.IDENTIFIER, lexeme='forfor'      , literal=None       , line=9)
Token(type=TokenType.CONST       , lexeme='const'       , literal=None       , line=9)
Token(type=TokenType.EOF         , lexeme=''            , literal=None       , line=9)

```

Actual Output: **Matches Expected Output**

```

PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC2-Multi_Line_Keywords.cn
Token(type=TokenType.IDENTIFIER, lexeme='andandand' , literal=None , line=1)
Token(type=TokenType.FALSE    , lexeme='false'    , literal=None , line=3)
Token(type=TokenType.TRUE     , lexeme='true'     , literal=None , line=3)
Token(type=TokenType.BREAK    , lexeme='break'    , literal=None , line=3)
Token(type=TokenType.RETURN   , lexeme='return'   , literal=None , line=5)
Token(type=TokenType.SUPER    , lexeme='super'    , literal=None , line=5)
Token(type=TokenType.FUN      , lexeme='fun'      , literal=None , line=5)
Token(type=TokenType.IF       , lexeme='if'       , literal=None , line=5)
Token(type=TokenType.ELSE     , lexeme='else'     , literal=None , line=5)
Token(type=TokenType.ELSE     , lexeme='else'     , literal=None , line=7)
Token(type=TokenType.IF       , lexeme='if'       , literal=None , line=7)
Token(type=TokenType.ELSEIF   , lexeme='elseif'   , literal=None , line=7)
Token(type=TokenType.NULL     , lexeme='null'     , literal=None , line=9)
Token(type=TokenType.THIS     , lexeme='this'     , literal=None , line=9)
Token(type=TokenType.IDENTIFIER, lexeme='forfor'   , literal=None , line=9)
Token(type=TokenType.CONST    , lexeme='const'    , literal=None , line=9)
Token(type=TokenType.EOF      , lexeme=''         , literal=None , line=9)

```

TC3 – Operators

- Ensure each operator scans to the correct token

Expected Output:

```

Token(type=TokenType.LEFT_PAREN , lexeme='(' , literal=None , line=1)
Token(type=TokenType.RIGHT_PAREN , lexeme=')' , literal=None , line=2)

```

Token(type=TokenType.LEFT_BRACE	, lexeme='{'	, literal=None	, line=3)
Token(type=TokenType.RIGHT_BRACE	, lexeme='}'	, literal=None	, line=4)
Token(type=TokenType.COMMA	, lexeme=','	, literal=None	, line=5)
Token(type=TokenType.DOT	, lexeme='.'	, literal=None	, line=6)
Token(type=TokenType.MINUS	, lexeme='-'	, literal=None	, line=7)
Token(type=TokenType.PLUS	, lexeme='+'	, literal=None	, line=8)
Token(type=TokenType.SEMICOLON	, lexeme=';'	, literal=None	, line=9)
Token(type=TokenType.STAR	, lexeme='*'	, literal=None	, line=10)
Token(type=TokenType.SLASH	, lexeme='/'	, literal=None	, line=11)
Token(type=TokenType.MODULO	, lexeme='%'	, literal=None	, line=12)
Token(type=TokenType.BANG	, lexeme='!'	, literal=None	, line=13)
Token(type=TokenType.EQUAL	, lexeme='='	, literal=None	, line=14)
Token(type=TokenType.EQUAL_EQUAL	, lexeme=='=='	, literal=None	, line=15)
Token(type=TokenType.BANG_EQUAL	, lexeme='!='	, literal=None	, line=16)
Token(type=TokenType.GREATER	, lexeme='>'	, literal=None	, line=17)
Token(type=TokenType.GREATER_EQUAL	, lexeme='>='	, literal=None	, line=18)
Token(type=TokenType.LESS	, lexeme='<'	, literal=None	, line=19)
Token(type=TokenType.LESS_EQUAL	, lexeme='<='	, literal=None	, line=20)
Token(type=TokenType.EOF	, lexeme=''	, literal=None	, line=20)

Actual Output: Matches Expected Output

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC3-Operators.cn
Token(type=TokenType.LEFT_PAREN, lexeme='(' , literal=None , line=1)
Token(type=TokenType.RIGHT_PAREN, lexeme=')' , literal=None , line=2)
Token(type=TokenType.LEFT_BRACE, lexeme='{ ' , literal=None , line=3)
Token(type=TokenType.RIGHT_BRACE, lexeme='}' , literal=None , line=4)
Token(type=TokenType.COMMA , lexeme=',' , literal=None , line=5)
Token(type=TokenType.DOT , lexeme='.' , literal=None , line=6)
Token(type=TokenType.MINUS , lexeme='-' , literal=None , line=7)
Token(type=TokenType.PLUS , lexeme='+' , literal=None , line=8)
Token(type=TokenType.SEMICOLON , lexeme=';' , literal=None , line=9)
Token(type=TokenType.STAR , lexeme='*' , literal=None , line=10)
Token(type=TokenType.SLASH , lexeme='/' , literal=None , line=11)
Token(type=TokenType.MODULO , lexeme='%' , literal=None , line=12)
Token(type=TokenType.BANG , lexeme='!' , literal=None , line=13)
Token(type=TokenType.EQUAL , lexeme='=' , literal=None , line=14)
Token(type=TokenType.EQUAL_EQUAL , lexeme=='==' , literal=None , line=15)
Token(type=TokenType.BANG_EQUAL , lexeme='!=' , literal=None , line=16)
Token(type=TokenType.GREATER , lexeme='>' , literal=None , line=17)
Token(type=TokenType.GREATER_EQUAL , lexeme='>=' , literal=None , line=18)
Token(type=TokenType.LESS , lexeme='<' , literal=None , line=19)
Token(type=TokenType.LESS_EQUAL , lexeme='<=' , literal=None , line=20)
Token(type=TokenType.EOF , lexeme='' , literal=None , line=20)
```

TC4 – Multi_Line_Operators

Purpose

- Ensure several operators on the same line are scanned appropriately
- Ensure operators are correctly scanned regardless of spacing
- Verify the double-character operators (ex. === scanning as EQUAL_EQUAL and EQUAL, rather than three EQUAL tokens)
- Verify that newlines and tabs are not consumed by the scanner and do not produce a token

Expected Output:

```
Token(type=TokenType.BANG , lexeme='!' , literal=None , line=1)
Token(type=TokenType.BANG_EQUAL , lexeme='!=' , literal=None , line=1)
Token(type=TokenType.EQUAL_EQUAL , lexeme=='==' , literal=None , line=1)
Token(type=TokenType.EQUAL , lexeme='=' , literal=None , line=1)
Token(type=TokenType.GREATER , lexeme='>' , literal=None , line=2)
Token(type=TokenType.GREATER_EQUAL , lexeme='>=' , literal=None , line=2)
Token(type=TokenType.GREATER , lexeme='>' , literal=None , line=2)
Token(type=TokenType.GREATER , lexeme='>' , literal=None , line=2)
Token(type=TokenType.EQUAL_EQUAL , lexeme=='==' , literal=None , line=2)
```

Token(type=TokenType.EQUAL	, lexeme='='	, literal=None	, line=2)
Token(type=TokenType.LESS	, lexeme='<'	, literal=None	, line=3)
Token(type=TokenType.LESS_EQUAL	, lexeme='<='	, literal=None	, line=3)
Token(type=TokenType.LESS_EQUAL	, lexeme='<='	, literal=None	, line=3)
Token(type=TokenType.LESS	, lexeme='<'	, literal=None	, line=3)
Token(type=TokenType.LESS	, lexeme='<'	, literal=None	, line=3)
Token(type=TokenType.MINUS	, lexeme='-'	, literal=None	, line=5)
Token(type=TokenType.MINUS	, lexeme='-'	, literal=None	, line=5)
Token(type=TokenType.PLUS	, lexeme='+'	, literal=None	, line=5)
Token(type=TokenType.PLUS	, lexeme='+'	, literal=None	, line=5)
Token(type=TokenType.SLASH	, lexeme='/'	, literal=None	, line=5)
Token(type=TokenType.SLASH	, lexeme='/'	, literal=None	, line=5)
Token(type=TokenType.BANG_EQUAL	, lexeme='!='	, literal=None	, line=7)
Token(type=TokenType.BANG_EQUAL	, lexeme='!='	, literal=None	, line=7)
Token(type=TokenType.BANG_EQUAL	, lexeme='!='	, literal=None	, line=7)
Token(type=TokenType.EOF	, lexeme=''	, literal=None	, line=7)

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC4-Multi_Line_Operators.cn
Token(type=TokenType.BANG, lexeme='!', literal=None, line=1)
Token(type=TokenType.BANG_EQUAL, lexeme='!=', literal=None, line=1)
Token(type=TokenType.EQUAL_EQUAL, lexeme=='==', literal=None, line=1)
Token(type=TokenType.EQUAL, lexeme='=', literal=None, line=1)
Token(type=TokenType.GREATER, lexeme='>', literal=None, line=2)
Token(type=TokenType.GREATER_EQUAL, lexeme='>=', literal=None, line=2)
Token(type=TokenType.GREATER, lexeme='>', literal=None, line=2)
Token(type=TokenType.GREATER, lexeme='>', literal=None, line=2)
Token(type=TokenType.EQUAL_EQUAL, lexeme=='==', literal=None, line=2)
Token(type=TokenType.EQUAL, lexeme='=', literal=None, line=2)
Token(type=TokenType.LESS, lexeme='<', literal=None, line=3)
Token(type=TokenType.LESS_EQUAL, lexeme='<=', literal=None, line=3)
Token(type=TokenType.LESS_EQUAL, lexeme='<=', literal=None, line=3)
Token(type=TokenType.LESS, lexeme='<', literal=None, line=3)
Token(type=TokenType.LESS, lexeme='<', literal=None, line=3)
Token(type=TokenType.MINUS, lexeme='-', literal=None, line=5)
Token(type=TokenType.MINUS, lexeme='-', literal=None, line=5)
Token(type=TokenType.PLUS, lexeme='+', literal=None, line=5)
Token(type=TokenType.PLUS, lexeme='+', literal=None, line=5)
Token(type=TokenType.SLASH, lexeme='/', literal=None, line=5)
Token(type=TokenType.SLASH, lexeme='/', literal=None, line=5)
Token(type=TokenType.BANG_EQUAL, lexeme='!=', literal=None, line=7)
Token(type=TokenType.BANG_EQUAL, lexeme='!=', literal=None, line=7)
Token(type=TokenType.BANG_EQUAL, lexeme='!=', literal=None, line=7)
Token(type=TokenType.EOF, lexeme='', literal=None, line=7)
```

TC5- String Literals

Purpose

- Test an empty string
- Ensure all characters including letters, numbers, special characters, symbols, and keywords inside of the string are evaluated as a string rather than other tokens
- Ensure multi-line strings and strings with tabs inside of them maintain the spacing inside the token
- Ensure an error is raised for an Unterminated String
- *The token's line number for a multi-line string will be the ending line, that is expected and how I intended to implement it

Expected Output:

[line 14] Error: Unterminated string.

```
Token(type=TokenType.STRING , lexeme="" , literal= , line=1)
Token(type=TokenType.STRING , lexeme="hello world multi line"
, literal=hello world multi line, line=4)

Token(type=TokenType.STRING , lexeme="ZZZzzz" , literal=ZZZzzz , line=5)
Token(type=TokenType.STRING , lexeme="123" , literal=123 , line=6)
Token(type=TokenType.STRING , lexeme="1.0" , literal=1.0 , line=7)
Token(type=TokenType.STRING , lexeme=".0" , literal=.0 , line=8)
Token(type=TokenType.STRING , lexeme="String with TABS in it...."
, literal=String with TABS in it..., line=9)

Token(type=TokenType.STRING , lexeme="string with keywords for modulo plus minus
else if var" , literal=string with keywords for modulo plus minus else if var , line=10)
Token(type=TokenType.STRING , lexeme="Test Special Chars $ % ^ & * * ( ) [ ] { } ! @ "
, literal=Test Special Chars $ % ^ & * * ( ) [ ] { } ! @ , line=11)

Token(type=TokenType.STRING , lexeme="123 + 987" , literal=123 + 987 , line=12)
Token(type=TokenType.STRING , lexeme="Other < Sybmols ? / \ | ~"
, literal=Other < Sybmols ? / \ | ~ , line=13)

Token(type=TokenType.EOF , lexeme="" , literal=None , line=14)
```

Actual Output: Matches Expected Output

```
[Line 14] Error: Unterminated string.
Token(type=TokenType.STRING , lexeme="" , literal= , line=1)
Token(type=TokenType.STRING , lexeme="hello world
multi line", literal=hello world
multi line, line=4)
Token(type=TokenType.STRING , lexeme="ZZZzzz" , literal=ZZZzzz , line=5)
Token(type=TokenType.STRING , lexeme="123" , literal=123 , line=6)
Token(type=TokenType.STRING , lexeme="1.0" , literal=1.0 , line=7)
Token(type=TokenType.STRING , lexeme=".0" , literal=.0 , line=8)
Token(type=TokenType.STRING , lexeme="String with TABS in it..." , literal=String with TABS in
it..., line=9)
Token(type=TokenType.STRING , lexeme="string with keywords for modulo plus minus else if var" , literal=string with
keywords for modulo plus minus else if var, line=10)
Token(type=TokenType.STRING , lexeme="Test Special Chars $ % ^ & * ( ) [ ] { } ! @" , literal=Test Special Chars $
% ^ & * ( ) [ ] { } ! @, line=11)
Token(type=TokenType.STRING , lexeme="123 + 987" , literal=123 + 987 , line=12)
Token(type=TokenType.STRING , lexeme="Other < Sybmols ? / \ | ~" , literal=Other < Sybmols ? / \ | ~, line=13)
Token(type=TokenType.EOF , lexeme="" , literal=None , line=14)
```

TC6 – Identifiers

Purpose

- Ensure valid identifiers can contain numbers, alphabet characters, and underscores
- Ensure symbols and special characters other than underscore are not included in the IDENTIFIER token

Expected Output:

[line 10] Error at character: ^: Unexpected character

[line 10] Error at character: :: Unexpected character

Token(type=TokenType.IDENTIFIER , lexeme='variable_name' , literal=None , line=1)

Token(type=TokenType.IDENTIFIER , lexeme='_var' , literal=None, line=2)

Token(type=TokenType.IDENTIFIER, lexeme='helloworld' , literal=None, line=3)

Token(type=TokenType.IDENTIFIER, lexeme='__123' , literal=None, line=4)

Token(type=TokenType.IDENTIFIER, lexeme='ABC_123_zzz' , literal=None, line=5)

Token(type=TokenType.NUMBER , lexeme='123' , literal=123.0, line=7)

Token(type=TokenType.IDENTIFIER, lexeme='abc ' , literal=None, line=7)

Token(type=TokenType.IDENTIFIER, lexeme='variable' , literal=None, line=8)

Token(type=TokenType.MINUS , lexeme='- ' , literal=None, line=8)

Token(type=TokenType.IDENTIFIER, lexeme='name' , literal=None, line=8)

```
Token(type=TokenType.IDENTIFIER, lexeme='invalid'           , literal=None, line=9)
Token(type=TokenType.DOT           , lexeme='.'           , literal=None, line=9)
Token(type=TokenType.IDENTIFIER, lexeme='name'           , literal=None, line=9)
Token(type=TokenType.STAR           , lexeme='*'           , literal=None, line=10)
Token(type=TokenType.IDENTIFIER , lexeme='not'           , literal=None, line=10)
Token(type=TokenType.IDENTIFIER , lexeme='an'            , literal=None, line=10)
Token(type=TokenType.IDENTIFIER, lexeme='identifier'     , literal=None, line=10)
Token(type=TokenType.EOF           , lexeme=''             , literal=None, line=10)
```

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC6-Identifiers.cn
[line 10] Error at character: ^: Unexpected character
[line 10] Error at character: :: Unexpected character
Token(type=TokenType.IDENTIFIER, lexeme='variable_name'     , literal=None, line=1)
Token(type=TokenType.IDENTIFIER, lexeme='_var'              , literal=None, line=2)
Token(type=TokenType.IDENTIFIER, lexeme='helloworld'        , literal=None, line=3)
Token(type=TokenType.IDENTIFIER, lexeme='__123'              , literal=None, line=4)
Token(type=TokenType.IDENTIFIER, lexeme='ABC_123_zzz'        , literal=None, line=5)
Token(type=TokenType.NUMBER    , lexeme='123'         , literal=123.0, line=7)
Token(type=TokenType.IDENTIFIER, lexeme='abc'         , literal=None, line=7)
Token(type=TokenType.IDENTIFIER, lexeme='variable'    , literal=None, line=8)
Token(type=TokenType.MINUS     , lexeme='-'          , literal=None, line=8)
Token(type=TokenType.IDENTIFIER, lexeme='name'       , literal=None, line=8)
Token(type=TokenType.IDENTIFIER, lexeme='invalid'    , literal=None, line=9)
Token(type=TokenType.DOT       , lexeme='.'          , literal=None, line=9)
Token(type=TokenType.IDENTIFIER, lexeme='name'       , literal=None, line=9)
Token(type=TokenType.STAR      , lexeme='*'          , literal=None, line=10)
Token(type=TokenType.IDENTIFIER, lexeme='not'        , literal=None, line=10)
Token(type=TokenType.IDENTIFIER, lexeme='an'         , literal=None, line=10)
Token(type=TokenType.IDENTIFIER, lexeme='identifier' , literal=None, line=10)
Token(type=TokenType.EOF       , lexeme=''           , literal=None, line=10)
```

TC7 – Number Literals

Purpose

- Ensure numbers output the correct token with the correct lexeme and literal
- Ensure .0 produce DOT and NUMBER and 0. case outputs NUMBER and DOT tokens

Expected Output:

```
Token(type=TokenType.NUMBER , lexeme='1'           , literal=1.0      , line=1)
Token(type=TokenType.NUMBER , lexeme='20'          , literal=20.0     , line=2)
Token(type=TokenType.NUMBER , lexeme='300'         , literal=300.0    , line=3)
```

```

Token(type=TokenType.NUMBER , lexeme='4000' , literal=4000.0 , line=4)
Token(type=TokenType.NUMBER , lexeme='999999' , literal=999999.0 , line=5)
Token(type=TokenType.NUMBER , lexeme='0' , literal=0.0 , line=6)
Token(type=TokenType.NUMBER , lexeme='0.0' , literal=0.0 , line=7)
Token(type=TokenType.NUMBER , lexeme='1.0' , literal=1.0 , line=8)
Token(type=TokenType.NUMBER , lexeme='12.99' , literal=12.99 , line=9)
Token(type=TokenType.NUMBER , lexeme='15.0000001' , literal=15.0000001 , line=10)
Token(type=TokenType.NUMBER , lexeme='0.000000000256' , literal=2.56e-10 , line=11)
Token(type=TokenType.DOT , lexeme='.' , literal=None , line=12)
Token(type=TokenType.NUMBER , lexeme='0' , literal=0.0 , line=12)
Token(type=TokenType.NUMBER , lexeme='0' , literal=0.0 , line=13)
Token(type=TokenType.DOT , lexeme='.' , literal=None , line=13)
Token(type=TokenType.EOF , lexeme='' , literal=None , line=13)

```

Actual Output: **Matches Expected Output**

```

PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC7-Number_Literals.cn
Token(type=TokenType.NUMBER , lexeme='1' , literal=1.0 , line=1)
Token(type=TokenType.NUMBER , lexeme='20' , literal=20.0 , line=2)
Token(type=TokenType.NUMBER , lexeme='300' , literal=300.0 , line=3)
Token(type=TokenType.NUMBER , lexeme='4000' , literal=4000.0 , line=4)
Token(type=TokenType.NUMBER , lexeme='999999' , literal=999999.0 , line=5)
Token(type=TokenType.NUMBER , lexeme='0' , literal=0.0 , line=6)
Token(type=TokenType.NUMBER , lexeme='0.0' , literal=0.0 , line=7)
Token(type=TokenType.NUMBER , lexeme='1.0' , literal=1.0 , line=8)
Token(type=TokenType.NUMBER , lexeme='12.99' , literal=12.99 , line=9)
Token(type=TokenType.NUMBER , lexeme='15.0000001' , literal=15.0000001 , line=10)
Token(type=TokenType.NUMBER , lexeme='0.000000000256' , literal=2.56e-10 , line=11)
Token(type=TokenType.DOT , lexeme='.' , literal=None , line=12)
Token(type=TokenType.NUMBER , lexeme='0' , literal=0.0 , line=12)
Token(type=TokenType.NUMBER , lexeme='0' , literal=0.0 , line=13)
Token(type=TokenType.DOT , lexeme='.' , literal=None , line=13)
Token(type=TokenType.EOF , lexeme='' , literal=None , line=13)

```

TC8 – Comment_Single_Line

Purpose

- Ensure single line comments beginning with # do not produce tokens

Expected Output:

Token(type=TokenType.STRING , lexeme="hello" , literal=hello , line=4)

Token(type=TokenType.EOF , lexeme="" , literal=None , line=4)

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC8-Comment_Single_Line.cn
Token(type=TokenType.STRING , lexeme="hello" , literal=hello , line=4)
Token(type=TokenType.EOF , lexeme="" , literal=None , line=4)
```

TC9 – Comment_Block

Purpose

- Ensure nested comments do not produce tokens for the contents inside of the levels of the comment
- Ensure that the scanner recognizes the beginning (/*) and the end (*/) of the nested comments
- Ensure nested comments beginning with /* but missing and ending */ raise an error

Expected Output:

[line 21] Error: Unterminated block comment.

Token(type=TokenType.STRING , lexeme="This is a string token" , literal=This is a string token , line=5)

Token(type=TokenType.STRING , lexeme="This is a string after the nested comment" , literal=This is a string after the nested comment , line=17)

Token(type=TokenType.EOF , lexeme="" , literal=None , line=21)

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC9-Comment_Block.cn
[line 21] Error: Unterminated block comment.
Token(type=TokenType.STRING , lexeme="This is a string token", literal=This is a string token, line=5)
Token(type=TokenType.STRING , lexeme="This is a string after the nested comment", literal=This is a string after the nested comment, line=17)
Token(type=TokenType.EOF , lexeme="" , literal=None , line=21)
```

TC10 -Empty_File

Purpose

- Ensure the empty file is ran, it should only produce an EOF token

Expected Result:

Token(type=TokenType.EOF, lexeme='', literal=None, line=1)

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC10-Empty_File.cn
Token(type=TokenType.EOF, lexeme='', literal=None, line=1)
```

TC11 – Token_Boundaries

Purpose

- Test operators touching identifiers and numbers
- Test keyword boundaries
- Test words with keywords inside of them

Expected Result:

Token(type=TokenType.IDENTIFIER	, lexeme='breakfast '	literal=None, line=1)
Token(type=TokenType.SUPER	, lexeme='superman'	literal=None, line=1)
Token(type=TokenType.IDENTIFIER	, lexeme='modulooo'	literal=None, line=1)
Token(type=TokenType.IDENTIFIER	, lexeme='andelseif'	literal=None, line=1)
Token(type=TokenType.IDENTIFIER	, lexeme='x'	literal=None, line=2)
Token(type=TokenType.EQUAL_EQUAL	, lexeme=='=='	literal=None, line=2)
Token(type=TokenType.NUMBER	, lexeme='10'	literal=10.0, line=2)
Token(type=TokenType.IDENTIFIER	, lexeme='y'	literal=None, line=3)
Token(type=TokenType.LESS_EQUAL	, lexeme='<='	literal=None, line=3)
Token(type=TokenType.IDENTIFIER	, lexeme='hello'	literal=None, line=3)
Token(type=TokenType.IDENTIFIER	, lexeme='z'	literal=None, line=4)
Token(type=TokenType.GREATER_EQUAL	, lexeme='>='	literal=None, line=4)
Token(type=TokenType.NUMBER	, lexeme='9'	literal=9.0, line=4)
Token(type=TokenType.IDENTIFIER	, lexeme='a'	literal=None, line=5)
Token(type=TokenType.BANG_EQUAL	, lexeme='!='	literal=None, line=5)
Token(type=TokenType.NUMBER	, lexeme='3'	literal=3.0, line=5)
Token(type=TokenType.IDENTIFIER	, lexeme='b'	literal=None, line=6)
Token(type=TokenType.MODULO	, lexeme='%'	literal=None, line=6)

Token(type=TokenType.IDENTIFIER	, lexeme='constant'	, literal=None, line=6)
Token(type=TokenType.VAR	, lexeme='var'	, literal=None, line=7)
Token(type=TokenType.IDENTIFIER	, lexeme='x'	, literal=None, line=7)
Token(type=TokenType.EQUAL	, lexeme='='	, literal=None, line=7)
Token(type=TokenType.STRING	, lexeme='\"'abc\"'	, literal=abc, line=7)
Token(type=TokenType.IDENTIFIER	, lexeme='truefalse'	, literal=None, line=8)
Token(type=TokenType.STRING	, lexeme='\"'hello\"'	, literal=hello, line=9)
Token(type=TokenType.NUMBER	, lexeme='123'	, literal=123.0, line=9)
Token(type=TokenType.MODULO	, lexeme='%'	, literal=None, line=9)
Token(type=TokenType.EOF	, lexeme=''	, literal=None, line=9)

Actual Output: **Matches Expected Output**

```
PS C:\Users\HP\Desktop\CNeumann_CIS343\CNeumann_CIS343> python src/cn.py test/Lab1/TC11-Token_Boundaries.cn
Token(type=TokenType.IDENTIFIER, lexeme='breakfast', literal=None, line=1)
Token(type=TokenType.IDENTIFIER, lexeme='superman', literal=None, line=1)
Token(type=TokenType.IDENTIFIER, lexeme='modulooo', literal=None, line=1)
Token(type=TokenType.IDENTIFIER, lexeme='andelseif', literal=None, line=1)
Token(type=TokenType.IDENTIFIER, lexeme='x', literal=None, line=2)
Token(type=TokenType.EQUAL_EQUAL, lexeme='==', literal=None, line=2)
Token(type=TokenType.NUMBER, lexeme='10', literal=10.0, line=2)
Token(type=TokenType.IDENTIFIER, lexeme='y', literal=None, line=3)
Token(type=TokenType.LESS_EQUAL, lexeme='<=', literal=None, line=3)
Token(type=TokenType.IDENTIFIER, lexeme='hello', literal=None, line=3)
Token(type=TokenType.IDENTIFIER, lexeme='z', literal=None, line=4)
Token(type=TokenType.GREATER_EQUAL, lexeme='>=', literal=None, line=4)
Token(type=TokenType.NUMBER, lexeme='9', literal=9.0, line=4)
Token(type=TokenType.IDENTIFIER, lexeme='a', literal=None, line=5)
Token(type=TokenType.BANG_EQUAL, lexeme='!=', literal=None, line=5)
Token(type=TokenType.NUMBER, lexeme='3', literal=3.0, line=5)
Token(type=TokenType.IDENTIFIER, lexeme='b', literal=None, line=6)
Token(type=TokenType.MODULO, lexeme='%', literal=None, line=6)
Token(type=TokenType.IDENTIFIER, lexeme='constant', literal=None, line=6)
Token(type=TokenType.VAR, lexeme='var', literal=None, line=7)
Token(type=TokenType.IDENTIFIER, lexeme='x', literal=None, line=7)
Token(type=TokenType.EQUAL, lexeme='=', literal=None, line=7)
Token(type=TokenType.STRING, lexeme='\"'abc\"', literal=abc, line=7)
Token(type=TokenType.IDENTIFIER, lexeme='truefalse', literal=None, line=8)
Token(type=TokenType.STRING, lexeme='\"'hello\"', literal=hello, line=9)
Token(type=TokenType.NUMBER, lexeme='123', literal=123.0, line=9)
Token(type=TokenType.MODULO, lexeme='%', literal=None, line=9)
Token(type=TokenType.EOF, lexeme='', literal=None, line=9)
```

TC12-Test_Errors

Purpose

- Ensure errors are raised for all unexpected characters
- Another test that error is raised for an unterminated string

Expected Result:

[line 1] Error at character: ^: Unexpected character

[line 1] Error at character: ?: Unexpected character

[line 1] Error at character: ~: Unexpected character

[line 1] Error at character: \$: Unexpected character

[line 1] Error at character: `: Unexpected character

[line 1] Error at character: &: Unexpected character

[line 1] Error at character: ': Unexpected character

[line 1] Error at character: :: Unexpected character

[line 1] Error at character: [: Unexpected character

[line 1] Error at character:]: Unexpected character

[line 1] Error at character: \: Unexpected character

[line 17] Error: Unterminated string.

Token(type=TokenType.IDENTIFIER, lexeme='test', literal=None, line=1)

Token(type=TokenType.IDENTIFIER, lexeme='unexpected', literal=None, line=1)

Token(type=TokenType.IDENTIFIER, lexeme='characters', literal=None, line=1)

Token(type=TokenType.EOF, lexeme='', literal=None, line=17)

Actual Result: **Matches Expected Output**

```
[line 1] Error at character: ^: Unexpected character
[line 1] Error at character: ?: Unexpected character
[line 1] Error at character: ~: Unexpected character
[line 1] Error at character: $: Unexpected character
[line 1] Error at character: `: Unexpected character
[line 1] Error at character: &: Unexpected character
[line 1] Error at character: ': Unexpected character
[line 1] Error at character: :: Unexpected character
[line 1] Error at character: [: Unexpected character
[line 1] Error at character: ]: Unexpected character
[line 1] Error at character: \: Unexpected character
[line 17] Error: Unterminated string.
Token(type=TokenType.IDENTIFIER, lexeme='test', literal=None, line=1)
Token(type=TokenType.IDENTIFIER, lexeme='unexpected', literal=None, line=1)
Token(type=TokenType.IDENTIFIER, lexeme='characters', literal=None, line=1)
Token(type=TokenType.EOF, lexeme='', literal=None, line=17)
```